

Solve these two equations for the partial derivatives, using

$$x_+ + x_- = -B \quad x_+ x_- = C \quad (9.5.25)$$

and find

$$\frac{\partial R}{\partial B} = BR_1 - S_1 \quad \frac{\partial S}{\partial B} = CR_1 \quad (9.5.26)$$

Bairstow's method now consists of using Newton-Raphson in two dimensions (which is actually the subject of the *next* section) to find a simultaneous zero of R and S . Synthetic division is used twice per cycle to evaluate R, S and their partial derivatives with respect to B, C . Like one-dimensional Newton-Raphson, the method works well in the vicinity of a root pair (real or complex), but it can fail miserably when started at a random point. We therefore recommend it only in the context of polishing tentative complex roots.

qroot.h

```
void qroot(VecDoub_I &p, Doub &b, Doub &c, const Doub eps)
```

Given $n+1$ coefficients $p[0..n]$ of a polynomial of degree n , and trial values for the coefficients of a quadratic factor x^2+bx+c , improve the solution until the coefficients b, c change by less than eps . The routine `poldiv` in §5.1 is used.

```
{
    const Int ITMAX=20;                At most ITMAX iterations.
    const Doub TINY=1.0e-14;
    Doub sc,sb,s,rc,rb,r,dv,delc,delb;
    Int n=p.size()-1;
    VecDoub d(3),q(n+1),qq(n+1),rem(n+1);
    d[2]=1.0;
    for (Int iter=0;iter<ITMAX;iter++) {
        d[1]=b;
        d[0]=c;
        poldiv(p,d,q,rem);
        s=rem[0];                        First division, r,s.
        r=rem[1];
        poldiv(q,d,qq,rem);
        sb = -c*(rc = -rem[1]);          Second division, partial r,s with respect to
        rb = -b*rc+(sc = -rem[0]);      c.
        dv=1.0/(sb*rc-sc*rb);           Solve 2x2 equation.
        delb=(r*sc-s*rc)*dv;
        delc=(-r*sb+s*rb)*dv;
        b += (delb=(r*sc-s*rc)*dv);
        c += (delc=(-r*sb+s*rb)*dv);
        if ((abs(delb) <= eps*abs(b) || abs(b) < TINY)
            && (abs(delc) <= eps*abs(c) || abs(c) < TINY)) {
            return;                      Coefficients converged.
        }
    }
    throw("Too many iterations in routine qroot");
}
```

We have already remarked on the annoyance of having two tentative roots collapse to one value under polishing. You are left not knowing whether your polishing procedure has lost a root, or whether there *is* actually a double root, which was split only by roundoff errors in your previous deflation. One solution is deflate-and-repolish; but deflation is what we are trying to avoid at the polishing stage. An alternative is *Maehly's procedure*. Maehly pointed out that the derivative of the reduced polynomial

$$P_j(x) \equiv \frac{P(x)}{(x - x_0) \cdots (x - x_{j-1})} \quad (9.5.27)$$

can be written as

$$P'_j(x) = \frac{P'(x)}{(x-x_0)\cdots(x-x_{j-1})} - \frac{P(x)}{(x-x_0)\cdots(x-x_{j-1})} \sum_{i=0}^{j-1} (x-x_i)^{-1} \quad (9.5.28)$$

Hence one step of Newton-Raphson, taking a guess x_k into a new guess x_{k+1} , can be written as

$$x_{k+1} = x_k - \frac{P(x_k)}{P'(x_k) - P(x_k) \sum_{i=0}^{j-1} (x_k - x_i)^{-1}} \quad (9.5.29)$$

This equation, if used with i ranging over the roots already polished, will prevent a tentative root from spuriously hopping to another one's true root. It is an example of so-called *zero suppression* as an alternative to true deflation.

Muller's method, which was described above, can also be a useful adjunct at the polishing stage.

CITED REFERENCES AND FURTHER READING:

- Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), Chapter 7.[1]
- Peters G., and Wilkinson, J.H. 1971, "Practical Problems Arising in the Solution of Polynomial Equations," *Journal of the Institute of Mathematics and Its Applications*, vol. 8, pp. 16–35.[2]
- IMSL Math/Library Users Manual (Houston: IMSL Inc.), see 2007+, <http://www.vni.com/products/ims1>. [3]
- Ralston, A., and Rabinowitz, P. 1978, *A First Course in Numerical Analysis*, 2nd ed.; reprinted 2001 (New York: Dover), §8.9–8.13.[4]
- Adams, D.A. 1967, "A Stopping Criterion for Polynomial Root Finding," *Communications of the ACM*, vol. 10, pp. 655–658.[5]
- Johnson, L.W., and Riess, R.D. 1982, *Numerical Analysis*, 2nd ed. (Reading, MA: Addison-Wesley), §4.4.3.[6]
- Henrici, P. 1974, *Applied and Computational Complex Analysis*, vol. 1 (New York: Wiley).
- Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §5.5 – §5.9.

9.6 Newton-Raphson Method for Nonlinear Systems of Equations

We make an extreme, but wholly defensible, statement: There are *no* good, general methods for solving systems of more than one nonlinear equation. Furthermore, it is not hard to see why (very likely) there *never will be* any good, general methods: Consider the case of two dimensions, where we want to solve simultaneously

$$\begin{aligned} f(x, y) &= 0 \\ g(x, y) &= 0 \end{aligned} \quad (9.6.1)$$

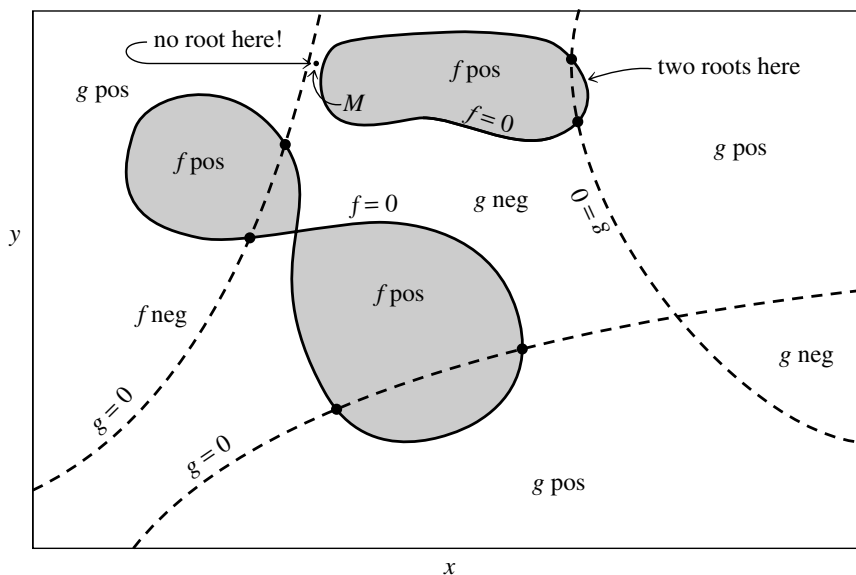


Figure 9.6.1. Solution of two nonlinear equations in two unknowns. Solid curves refer to $f(x, y)$, dashed curves to $g(x, y)$. Each equation divides the (x, y) -plane into positive and negative regions, bounded by zero curves. The desired solutions are the intersections of these unrelated zero curves. The number of solutions is a priori unknown.

The functions f and g are two arbitrary functions, each of which has zero contour lines that divide the (x, y) -plane into regions where their respective function is positive or negative. These zero contour boundaries are of interest to us. The solutions that we seek are those points (if any) that are common to the zero contours of f and g (see Figure 9.6.1). Unfortunately, the functions f and g have, in general, no relation to each other at all! There is nothing special about a common point from either f 's point of view, or from g 's. In order to find all common points, which are the solutions of our nonlinear equations, we will (in general) have to do neither more nor less than map out the full zero contours of both functions. Note further that the zero contours will (in general) consist of an unknown number of disjoint closed curves. How can we ever hope to know when we have found all such disjoint pieces?

For problems in more than two dimensions, we need to find points mutually common to N unrelated zero-contour hypersurfaces, each of dimension $N - 1$. You see that root finding becomes virtually impossible without insight! You will almost always have to use additional information, specific to your particular problem, to answer such basic questions as, “Do I expect a unique solution?” and “Approximately where?” Acton [1] has a good discussion of some of the particular strategies that can be tried.

In this section we discuss the simplest multidimensional root-finding method, Newton-Raphson. This method gives a very efficient means of converging to a root, if you have a sufficiently good initial guess. It can also spectacularly fail to converge, indicating (though not proving) that your putative root does not exist nearby. In §9.7 we discuss more sophisticated implementations of the Newton-Raphson method, which try to improve on Newton-Raphson's poor global convergence. A multidimensional generalization of the secant method, called Broyden's method, is also